File Management and Processing (IS321)

Academic Year 2026/2027 - Assignment 1

❖ System Overview

In this assignment, you will implement a simple Healthcare Management System in which an administrator can manage doctors and their appointments.

- You must store data about patient medical records and appointments. For simplicity, assume that each doctor can have many appointments and the appointment is handled by only one doctor.
- Your initial requirement is to store 5 doctors and 10 appointments.

❖ Storage and File Format

- **Binary Format:** Data must be stored in the file in **binary format**.
- **Record Structure:** Save the data for doctors and appointments using **fixed-length records**. Each field must be allocated exactly its specified size (padded with spaces or null characters if the actual data is shorter).
- **Header Record:** The start of the **AVAIL LIST** (the offset/reference to the first logically deleted record) must be stored at the **Header Record** at the very beginning of the data file as an integer.

❖ Data Files Specifications

Consider the following specifications for the fields of each record in both data files. *(Note: Fixed sizes mean Doctors records are exactly 95 bytes, and Appointments are exactly 60 bytes)*.

Doctors Data file (Total Record Size: 95 bytes)

- Char Doctor ID [15] // Primary key
- Char Doctor Name [30] // Secondary key
- Char Doctor Specialty [20] // Secondary key
- Char Address [30] // Non-Key Attribute

Appointments Data file (Total Record Size: 60 bytes)

- Char Appointment ID [15] // Primary key
- Char Appointment Date [30] // Non-Key Attribute
- Char Doctor ID [15] // Secondary Key (Foreign Key)

❖ Indexing Requirements

Please build the following indexes in memory/file:

1. Primary index using the Doctor ID (for Doctors data file).
2. Primary index using the Appointment ID (for Appointments data file).
3. Secondary index using the Doctor ID (for the Appointments data file).
4. Secondary index using the Doctor Name (for the Doctors data file).
5. Secondary index using the Doctor Specialty (for the Doctors data file).

Index Implementation Details:

- All indexes must be **sorted ascending**.
- Searching in indexes must be performed using **Binary Search**.
- Bind all secondary indexes with the primary index (do not bind them by byte addresses directly).
- You must implement secondary indexes using the **linked list on a different file technique** (any pointers or record links used here should be stored as integers, taking sizeof(int)).

❖ Operations & Maintenance

Implement add, update, and delete functions considering the following situations:

- **Add:** When you add a record, first look at the **AVAIL LIST** (read from the header record). If there is a record available in the AVAIL LIST, write the new record to the first available location the AVAIL LIST points to, and make appropriate changes to the AVAIL LIST to ensure its correctness.. If the list is empty, append to the end. If the record to be added already exists, do not write that record to the file.
- **Delete:** When you delete a record, **do not physically delete** it from the file. Just put a marker (*) on the record in the file, and make appropriate changes to the AVAIL LIST and Header Record. If the record to be deleted does not exist, display a warning message. Note: all add and delete operations will affect indexes.
- **Update (Non-Key):** Make updates to non-key fields in place, ensuring updates to these fields do not exceed the allocated size.
- **Update (Key):** If an update is made to a primary key (e.g., Doctor ID), **shifting must be used** in the primary index array to solve the rearrangement, keeping the index sorted properly after the key changes.

❖ Query Engine

The user can write a query that contains specific keywords. Your program should parse it and use the appropriate indexes.

Examples of user queries:

- Select all from Doctors where Doctor ID='xxx'; *(uses primary index)*
- Select all from Appointments where Doctor ID='xxx'; *(uses secondary index)*
- Select Doctor Name from Doctors where Doctor Name='xxx'; *(uses secondary index)*
- Select all from Doctors where Doctor Name='xxx' and Doctor Specialty='yyy'; *(uses multiple secondary indices).*

❖ Main Welcome Screen

6. Add New Doctor
7. Add New Appointment
8. Update Doctor Name (Doctor ID)
9. Update Doctor ID [Key Update Test]
10. Update Appointment Date (Appointment ID)
11. Delete Appointment (Appointment ID)
12. Delete Doctor (Doctor ID)
13. Print Doctor Info (Doctor ID)
14. Print Appointment Info (Appointment ID)
15. Write Query
16. Exit

❖ Samples of Data Stored (Binary/Fixed-Length Format Trace)

Note: While data is stored in binary format (e.g., ios::binary in C++), the trace below uses textual representations for readability. Fields are padded with spaces to match their exact char array sizes. The header contains a 4-byte integer (sizeof(int)) representing the byte offset of the Avail List head (initially -1).

1. Initial File State (Empty)

[Header (int): -1]

2. After Operation: Add Doctor (D1, Ali, Cardio, Cairo)

[Header (int): -1]
[D1][Ali][Cardio][Cairo] *(Total 95 bytes)*

3. After Operation: Add Doctor (D2, Omar, Neuro, Giza)

[Header (int): -1]

[D1][Ali][Cardio][Cairo]

[D2][Omar][Neuro][Giza]

4. After Operation: Delete Doctor (D1)

(Assume D1 record starts at byte offset 4 after the 4-byte header)

[Header (int): 4]

[*1][Ali][Cardio][Cairo] *(Replaced 'D' with marker '*'. Record logically deleted and added to AVAIL LIST)*

[D2][Omar][Neuro][Giza]

5. After Operation: Add Doctor (D3, Mona, Derma, Alex)

(System checks header, sees offset 4 is available. Overwrites deleted space, updates header back to -1)

[Header (int): -1]

[D3][Mona][Derma][Alex] *(Overwrites D1 space)*

[D2][Omar][Neuro][Giza]

Assignment Instructions

*o The deadline for this assignment is **Friday 13-11-2025***

o Each TA will announce the discussion time by the end of week starting 14-11-2025

o You are required to form a group of 5 students from the same lab or with the same TA.

o Cheating will be penalized with negative points.

o All team members will be discussed.

o NO late submission is accepted.

o Attach a cover page with the assignment file containing the students' names, IDs and TA-name.

o The assignment file should be named as: LeaderID_TAname_Assignment1.

o Only one team member will submit the assignment.

o The weight of this assignment is 10 points.